

Tema 5: Cues de prioritat i arbres generals

5.1 La cua de prioritat

A diferència d'una cua normal FIFO (el primer a entrar és el primer a sortir), una **cua de prioritat** atén sempre l'element **més gran o urgent** de tots els que esperen, sense importar quan ha entrat.

Si ho fem en un simple vector ordenat, el **push** és absurdament lent $\mathcal{O}(N)$ movent tots els elements. I si la fem desordenada, el **pop** tardarà $\mathcal{O}(N)$ en buscar on amagàvem el màxim. Necessitem portar el cost d'ambdues accions a un curt $\mathcal{O}(\log N)$. Quina és l'eina definitiva? El **binary heap**.

5.2 El binary heap

És un **arbre binari complet** modelat a dins d'un *simple vector pla*. L'arrel sempre és l'element màxim absolut de tota l'estructura.

L'arbre ignora la posició 0. L'arrel dominant viu coronada en la posició 1.

Per a qualsevol node a la posició i : - **Pare:** $i / 2$ - **Fill Esquerre:** $2 * i$ - **Fill Dret:** $2 * i + 1$

[Gràfic interactiu disponible a la versió web de l'apunt]

No calen punters ni regles complexes, tot és aritmètica fulgurant $\mathcal{O}(1)$.

push() - inserir i pujar (flow up)

L'element entra per l'última posició del vector. Llavors s'avalua de baix a dalt: *Soc més gran que el meu cap?* Si sí, intercanvi (**swap**) de posició amb ell cap amunt, iterant la pujada fins trobar el seu lloc jeràrquic. Exemple:

[Simulació interactiva disponible a la web]

pop() - extreure i baixar (flow down)

El rei (posició 1) ha sortit en ser processat. Per substituir-lo, prenem el “peó” (últim de tots al vector) i el plantem a la casella 1 de l'arrel. Llavors avalua de dalt a baix contra els dos fills: *Quin dels meus dos nous súbdits inferiors és més gran que jo?* Intercanvi de posició amb el fill **més gran**, iterant cap avall de cop baixant al forat que es mereix.

[Simulació interactiva disponible a la web]

Resum de temps heap: **top()** obté l'arrel en l'immediat $\mathcal{O}(1)$. Les insercions i neteges modifiquen l'altitud de l'arbre, requerint només salts de cost $\mathcal{O}(\log N)$.

5.3 Customitzant elements: l'operador >

Sovint els teus elements a examinar seran tuples o customitzacions com la pròpia **struct Persona**. Donat que tu li demanes al Heap que *col · loqui el màxim a dalt*, hauràs de donar regles de joc (C++) on re-defineixis

literalment l'operador MÉS GRAN (>) entre dues persones en l'ambient global.

```
struct Persona {
    string nom;
    int prioritat;
};

// Sobre-escrivim la lògica de "més gran" de C++
bool operator>(const Persona& a, const Persona& b) {
    return a.prioritat > b.prioritat;
}
```

Al main:: un cop fem Heap<Persona> cua; el directiu intern ja sap classificar VIPs ràpid per l'edat o puntaje pre-carregat!

5.4 Arbres generals (Tree<T>)

S'ha acabat limitar-se al tancat esquerre/dret. A un Tree<T>, un node Pare pot albergar un nombre arbitrari de nodes fill infinit. Un “**vector**” literal de fills.

Això canvia tota la matriu de recerca. L'etapa recursiva de cridar el t.left() i t.right() es modifica completament adoptant un **bucle iteratiu sobre les pertinences t.child(i)**, però la resta d'artificis recursius i immutables resten exactes.

La classe té la següent organització visible a la pública: - Tree(): creador estàndard buit absolut. - Tree(const T& val): crea la coronació (fulla única base del valor sense fills). - Tree(const T& val, vector<Tree> fills): Crea arrel assignant un valor i injectant una ramificació múltiple de cop fets gràcies a un vector inicial pre-carregat.

Consultores pràctiques per la nostra recursió: 1. value(): per llegir el cor de l'arrel. 2. num_children(): per la condició topall al límit iteratiu (N fills). 3. child(i): desplaçament en el vector intern extraient cada node pare per reiniciar la trucada de salt.

Buscar profundament en un general

S'itera la quantitat de fills de 0 a limit-1. En cas d'un encert interior retornem el booleà a la cadena superior parant tota dispersió erràtica a prop d'un $\mathcal{O}(N)$ salvador. No esperem que tot acabi com al recurs de dues ales antic, al trobar la clau curt-circuitem l'estratègia amb un True contundent que desapila sense remordiments cap arrel principal garantint prestacions robustes i impecables al sistema d'objectius moderns de dades.

[Simulació interactiva disponible a la web]